Algorithms

Lecture # 34

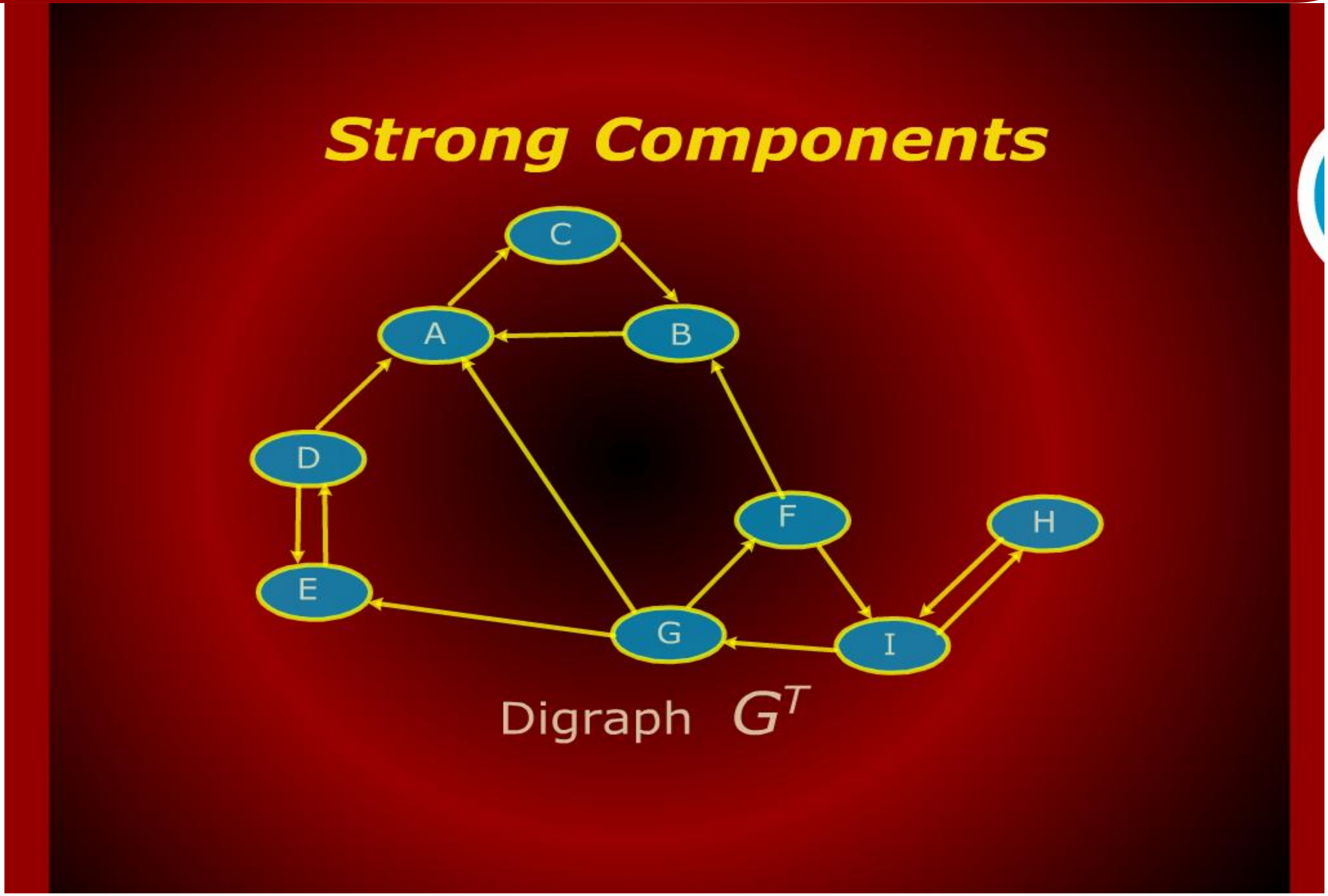
Syed Hamed Raza

Strong Components

Strong Components and DFS

- We will discuss the algorithm; the complete proof is in CLR.
- Define G^T to be the digraph with the same vertex set at G but in which all edges have been reversed in direction.
- Given an adjacency list for G, it is possible to compute G^T in $\Theta(V + E)$ time.

Strong Components



Strong Components and DFS

Strong Components

Strong Components

STRONG COMPONENTS(G)

1 Run DFS(G) computing finish times f[u]

2 Compute G^T

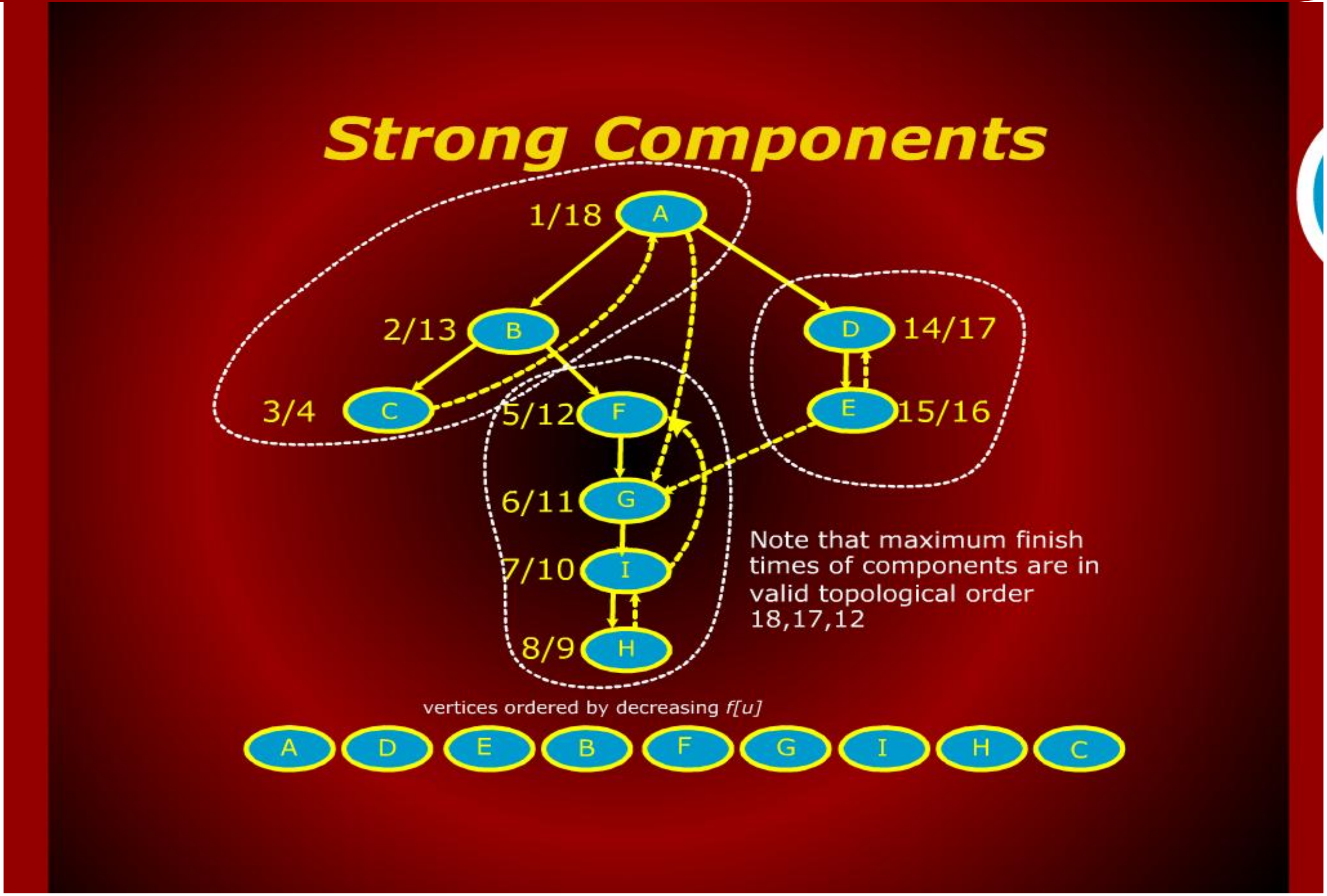
3 Sort vertices of G^T in decreasing f[u]

4 Run DFS(G^T) using this order

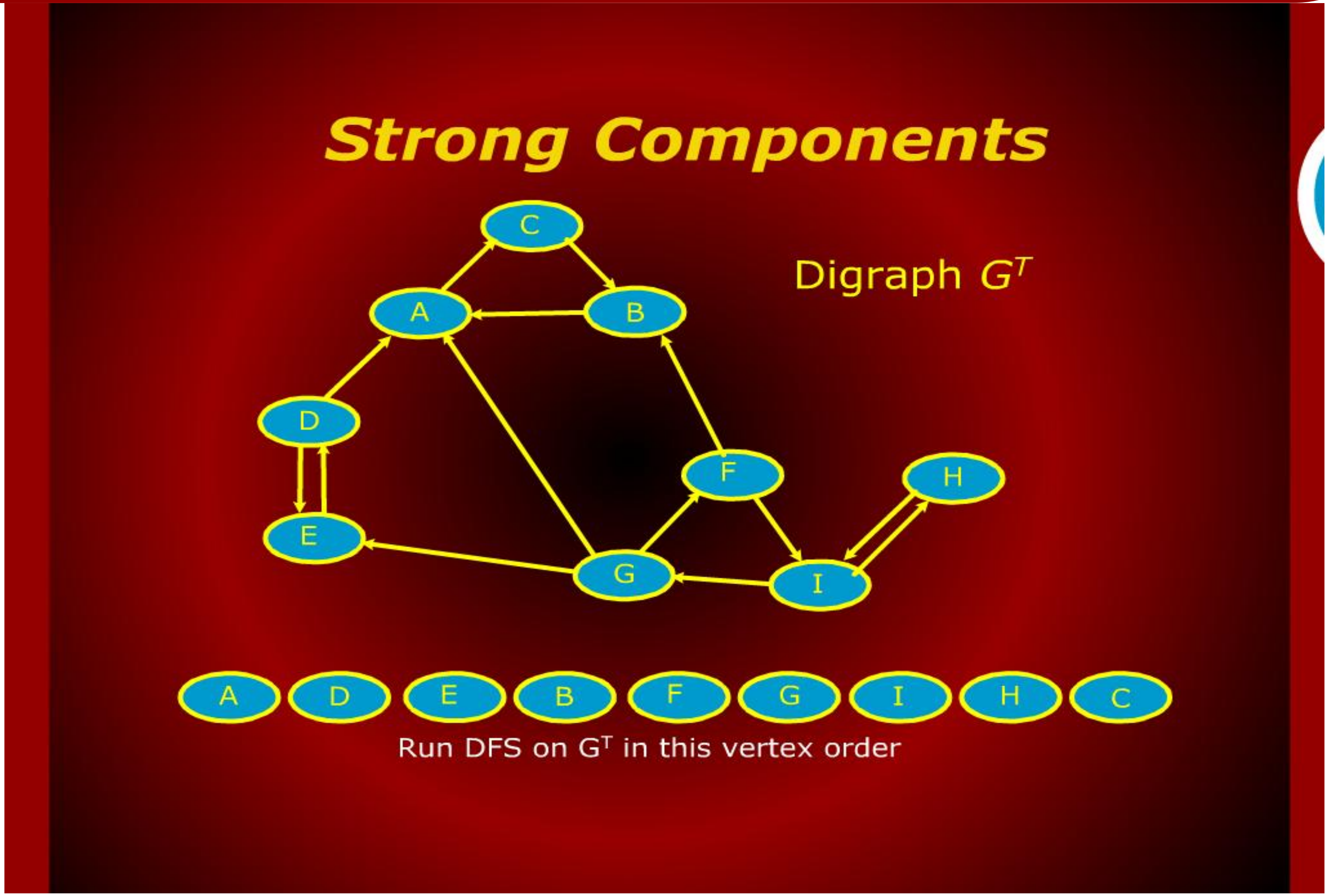
5 Each DFS tree is a strong component

All steps operate in $\Theta(V + E)$ time.

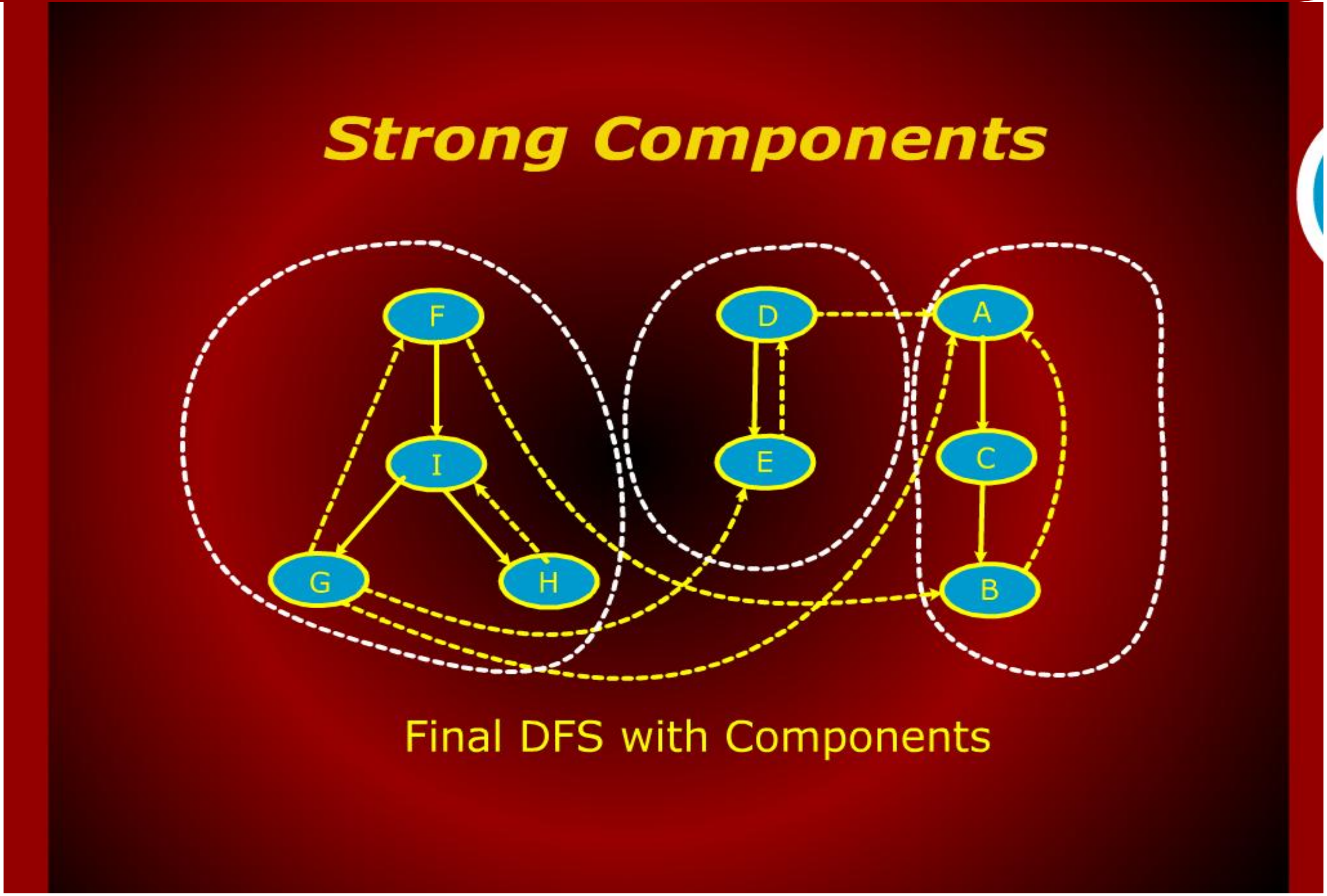
Strong Components



Strong Components



Strong Components



Minimum Spanning Trees

Minimum Spanning Trees

Minimum Spanning Trees

- A common problem is communications networks and circuit design is that of connecting together a set of nodes by a network of total minimum length.
- The length is the sum of lengths of connecting wires.
- Consider, for example, laying cable in a city for cable t.v.

Minimum Spanning Trees

Minimum Spanning Trees

- The computational problem is called the minimum spanning tree (MST) problem.
- Formally, we are given a connected, undirected graph G = (V, E)
- Each edge (u, v) has numeric weight of cost.

Minimum Spanning Trees

Minimum Spanning Trees

- We define the cost of a spanning tree T to be the sum of the costs of edges in the spanning tree

$$w(T) = \sum_{(u,v)\in T} w(u, v)$$

- A minimum spanning tree is a tree of minimum weight.

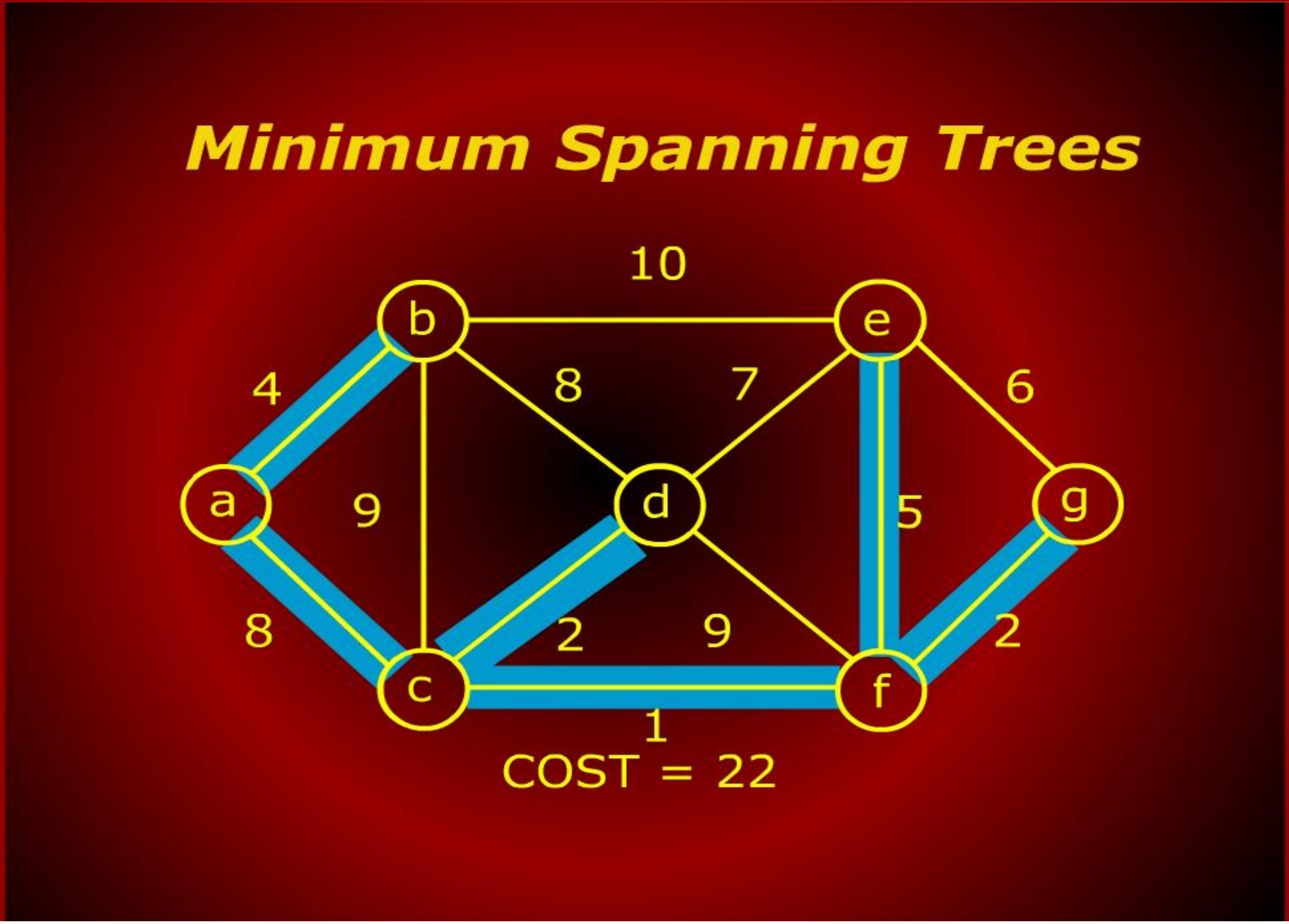
Minimum Spanning Trees

Minimum Spanning Trees

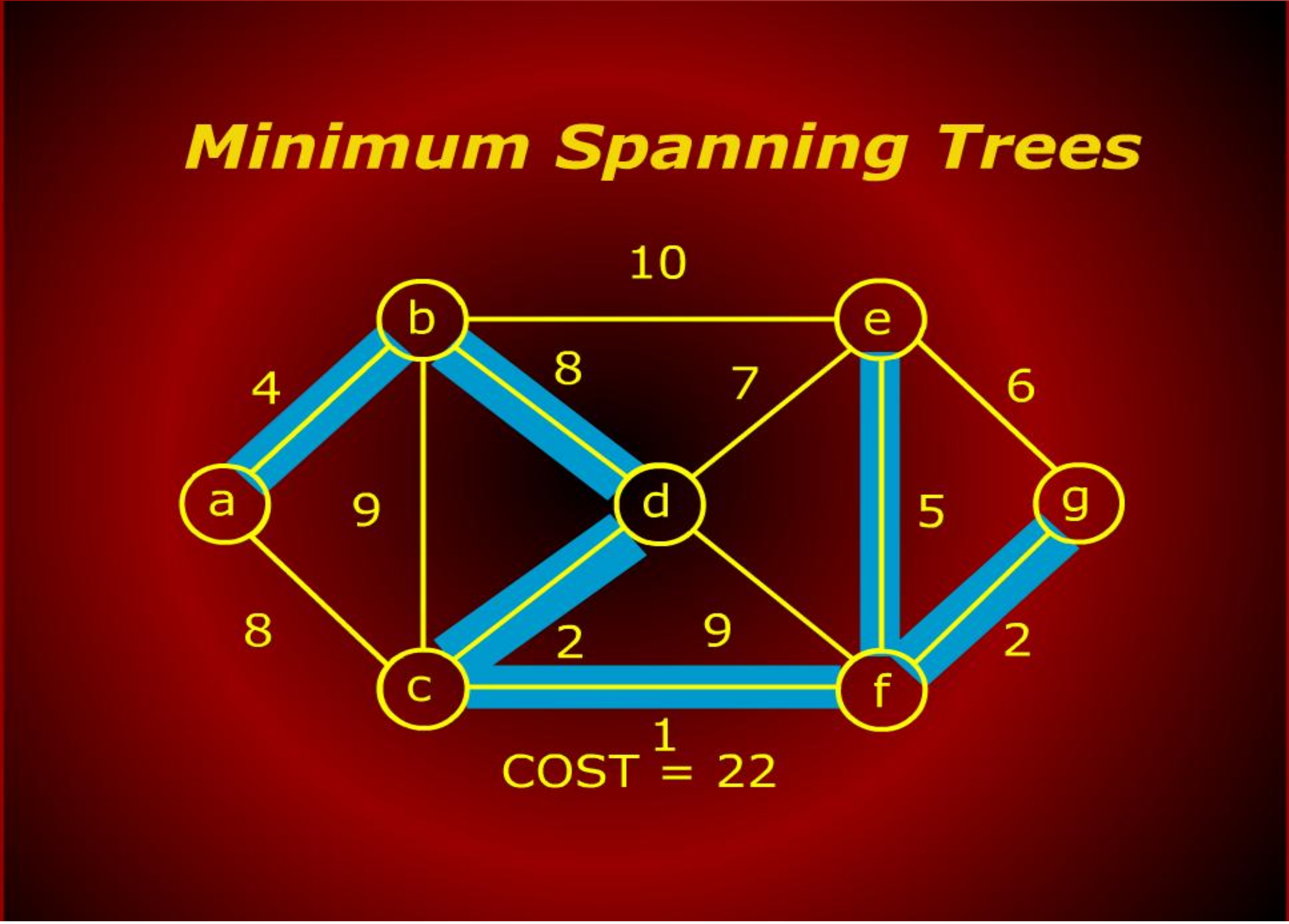
- The next three figures show three spanning trees for the same graph.
- The first is not a MST; the other two are.

Minimum Spanning Trees

Minimum Spanning Trees



Minimum Spanning Trees



Generic Approach

Generic Approach

- We will present two *greedy* algorithms (Kruskal's and Prim's) for computing MST.
- Recall that a greedy algorithm is one that builds a solution by repeatedly selecting the cheapest among all options at each stage.
- Once the choice is made, it is never undone.

Free Tree Facts

Free Tree Facts

Before presenting the two algorithms, let us review facts about free trees:

- A free tree with n vertices has exactly n - 1 edges.
- There exists a unique path between any two vertices of a free tree.
- Adding any edge to a free tree creates a unique cycle. Breaking any edge on this cycle restores the free tree.